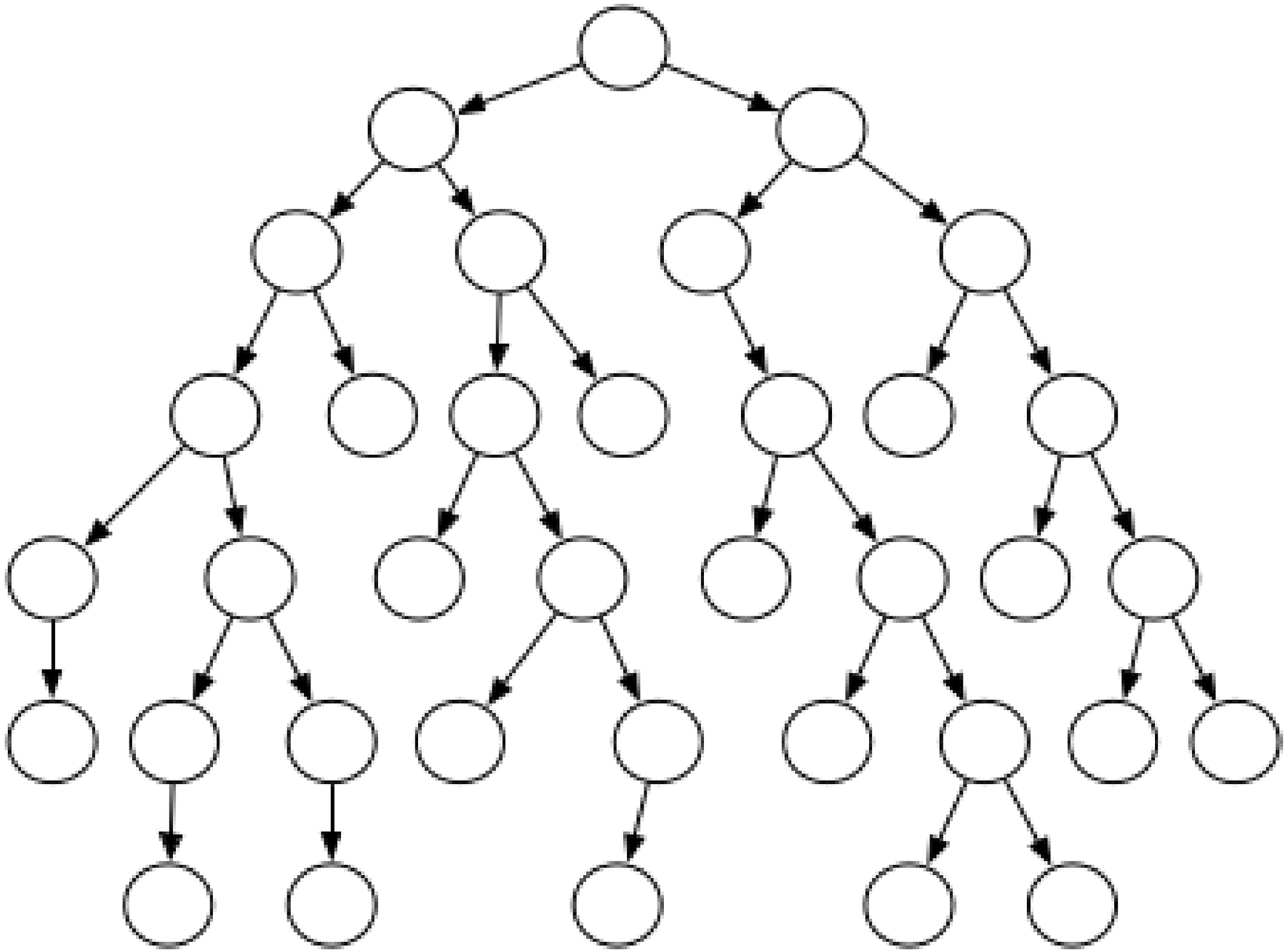
UCS3461 Foundations of Artificial Intelligence

MEMORY-BOUNDED SEARCH

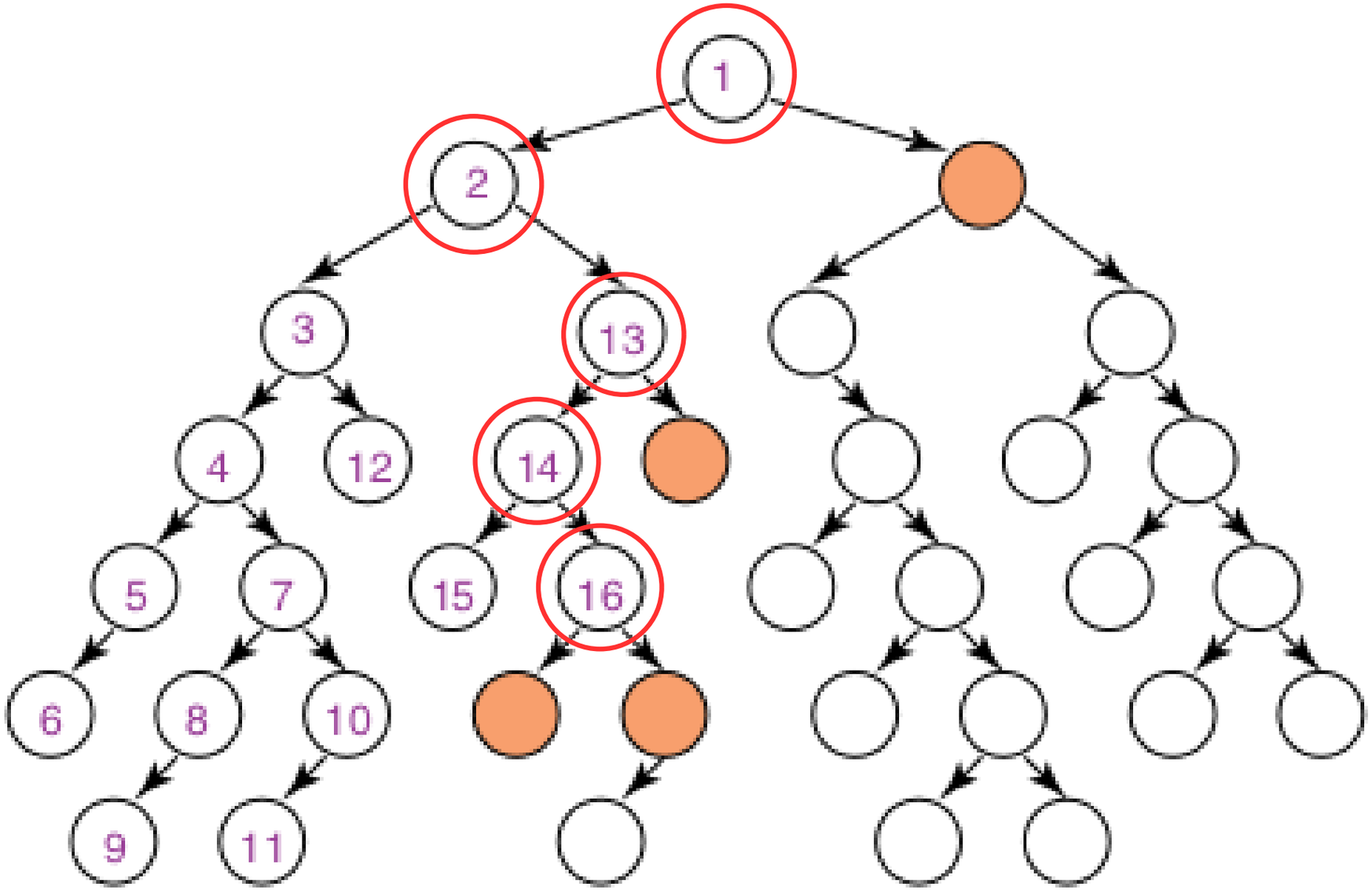
Semester 5, B.E. Computer Science and Engineering
2 January 2026

Sakaya Milton R
SSN College of Engineering

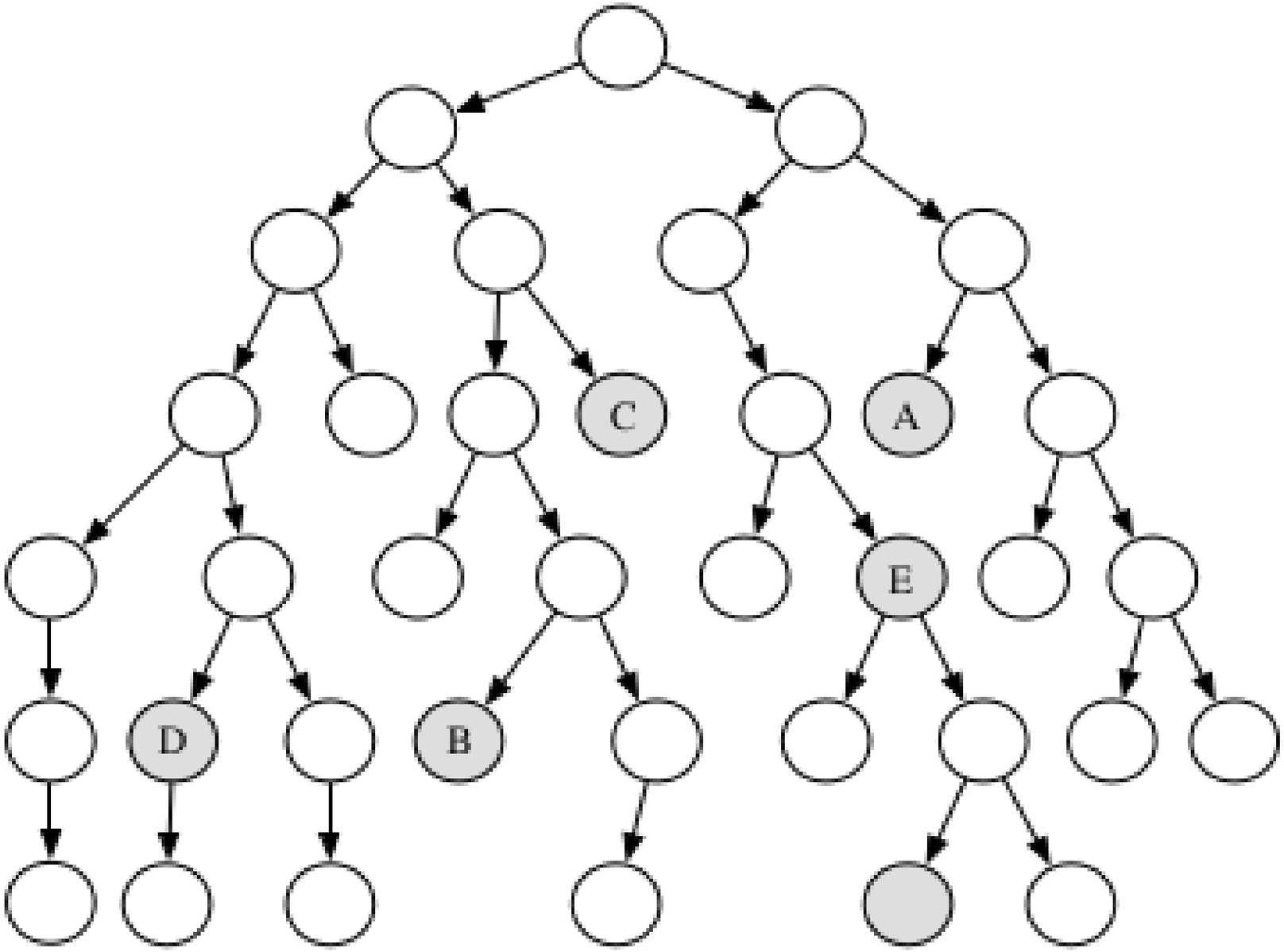
DFS Illustrative Graph



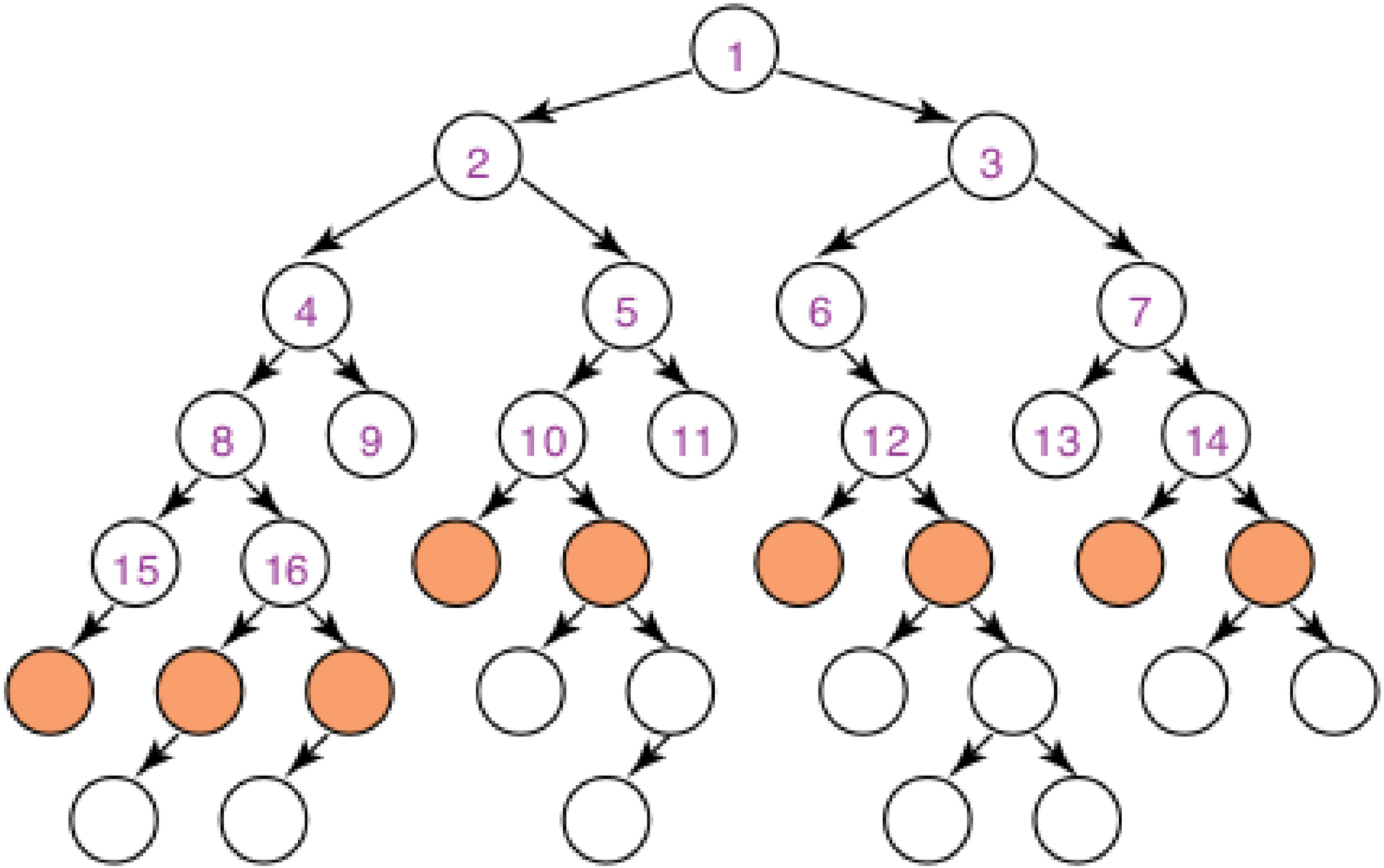
DFS Frontier and Live Path



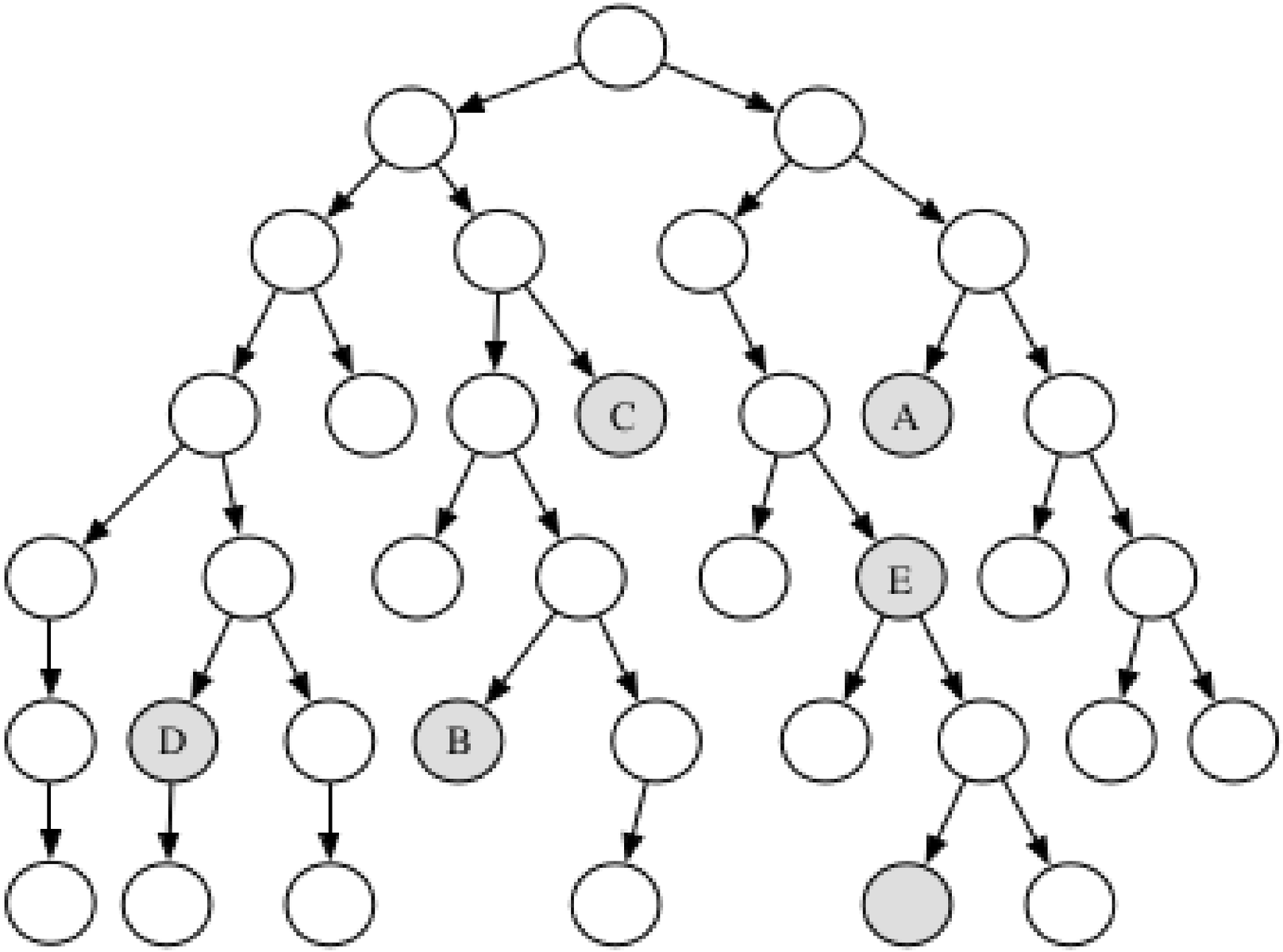
Which Shaded Node DFS Finds First?



BFS Frontier



Which Shaded Node BFS Finds First?



DFS (Depth-First Search)

Depth First Search (DFS) – Exhaustive Search

Algorithm: DFS (G, [s...,v], goal)

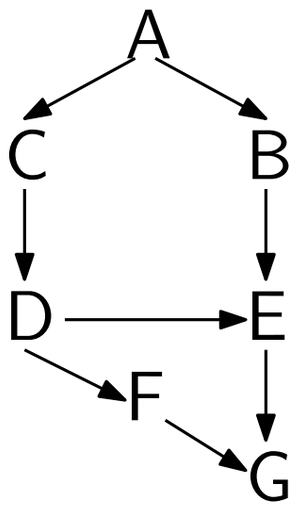
Input: G: graph
[s...,v]: path
goal: Boolean function on nodes

Output: path that extends [s...,v] to a goal node or failure if there is no such path.

```
if goal(v) then  return [s...,v]
foreach vw do
    result ← DF_Search(G, [s...,v,w], goal)
    if result ≠ failure then
        return result
return failure
```

- Do not test whether a vertex is explored – no explored set.
- No multiple-path pruning.

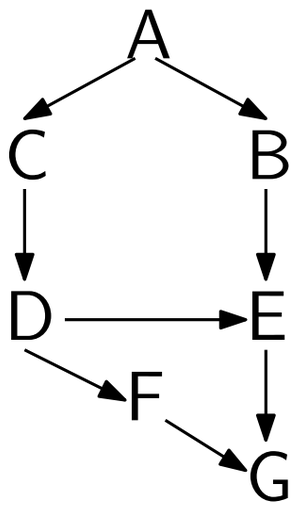
DFS Trace



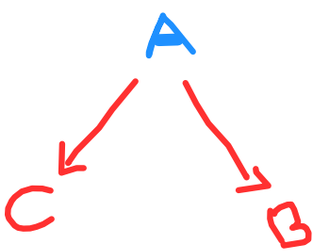
E F

A

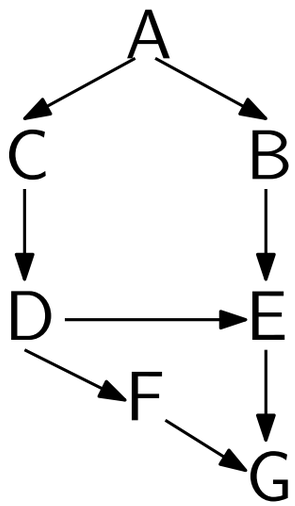
DFS Trace



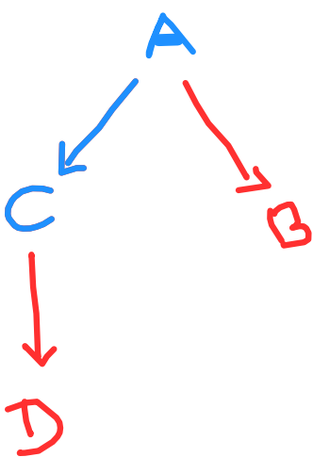
E	F
	– A
– A	AB, AC



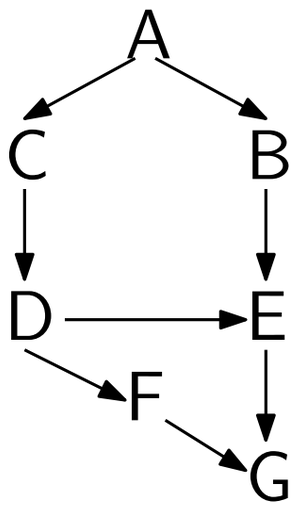
DFS Trace



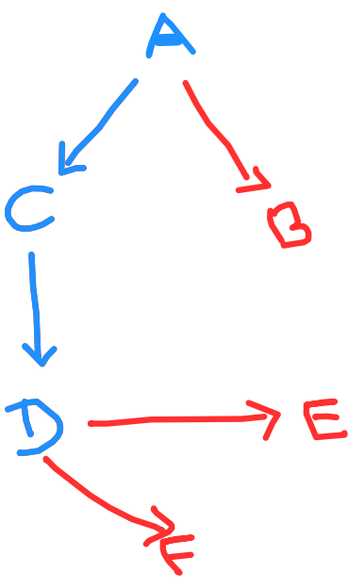
E	F
	– A
– A	AB, AC
AC	AB | CD



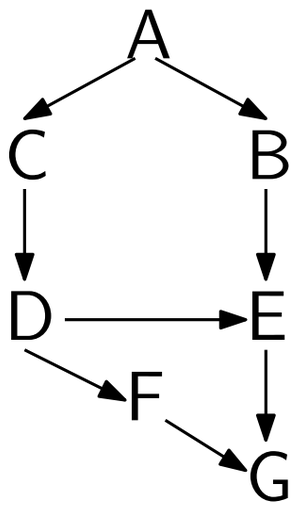
DFS Trace



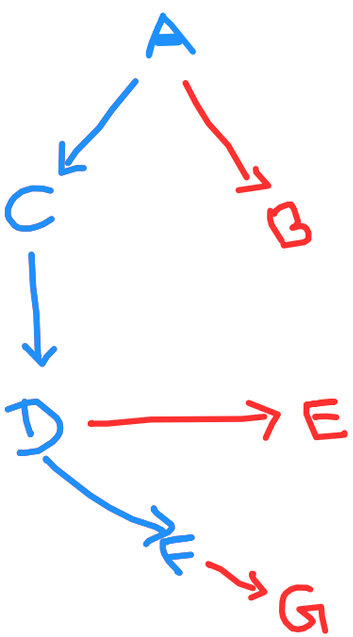
E	F
	– A
– A	AB, AC
AC	AB | CD
CD	AB | DE, DF



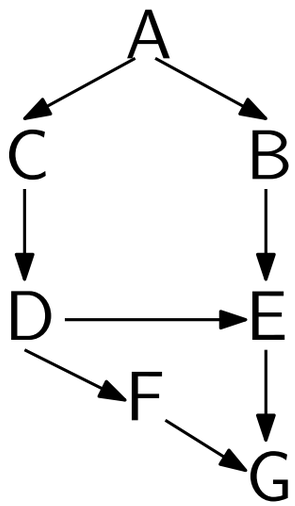
DFS Trace



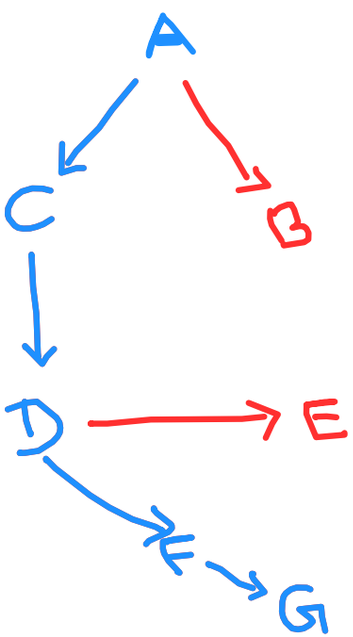
E	F
	– A
– A	AB, AC
AC	AB | CD
CD	AB | DE, DF
DF	AB | DE | FG



DFS Trace



E	F
	– A
– A	AB, AC
AC	AB | CD
CD	AB | DE, DF
DF	AB | DE | FG
FG	AB | DE



Depth-Limited DFS

Depth-Limited DFS

- A limited/bounded depth-first search takes a limit/bound (max cost or max depth) and does not expand paths that exceed the limit/bound.
  - explores part of the search graph,
  - uses space linear in the depth of the search.
- Never explore paths longer than depth limit.
- Where/when to test for goal?
  - At every level?
  - Only at the depth-limit level!

Depth-Limited DFS

Algorithm: Depth_Limited_DFS([s...v], limit)

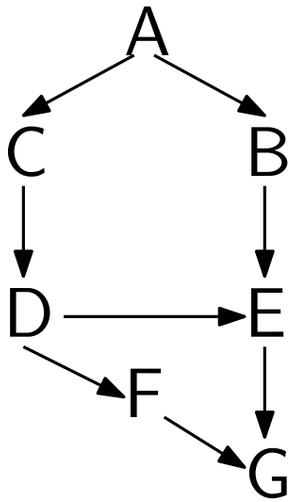
Input: [s..v]: path
limit ≥ 0

Output: path to goal, of length LIMIT

```
if limit > 0 then
    foreach vw do
        result ← Depth_Limited_DFS([s...,v,w], limit - 1)
        if result ≠ failure then
            return result
else if goal(v) then // why after limit > 0 test
    return [s...v]
else if v has any successors then
    depth_limit_hit? ← true
return failure
```

- Returns only a path of length LIMIT.

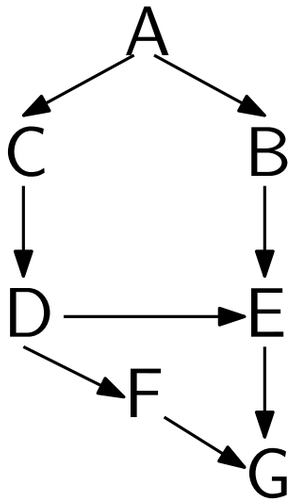
DL-DFS Tracing



E F

– A

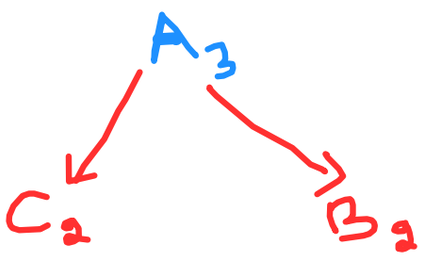
DL-DFS Tracing



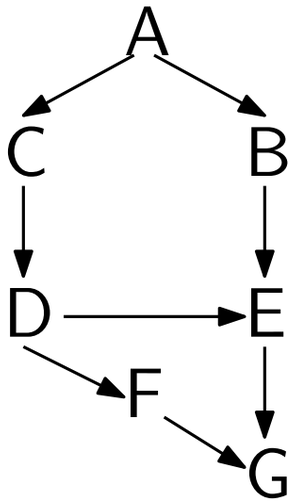
E F

$-A$

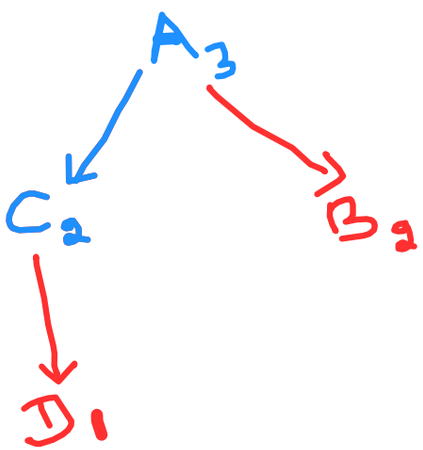
$-A_3$ AB_2, AC_2



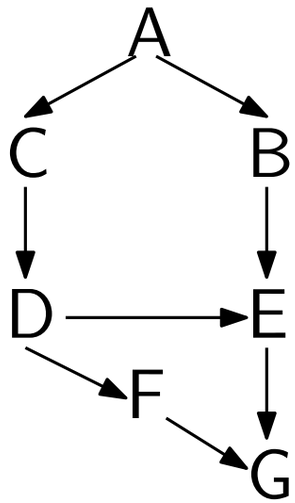
DL-DFS Tracing



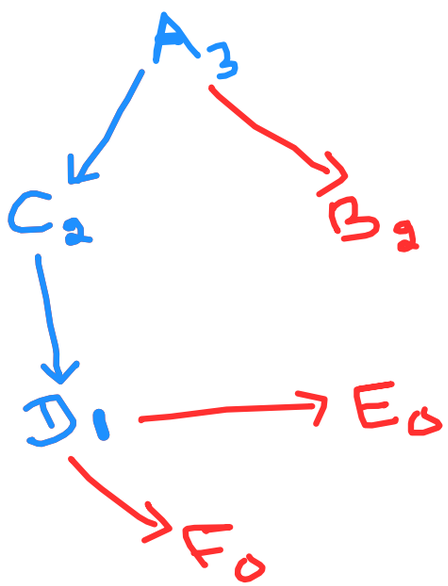
E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$



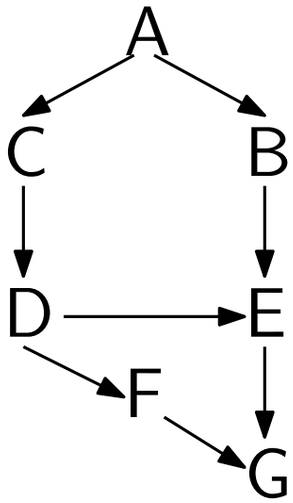
DL-DFS Tracing



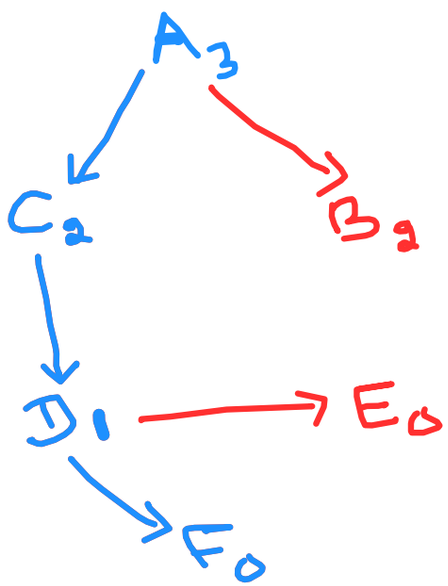
E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$
CD_1	$AB_2 \mid DE_0, DF_0$



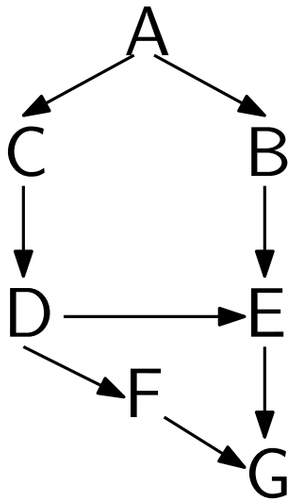
DL-DFS Tracing



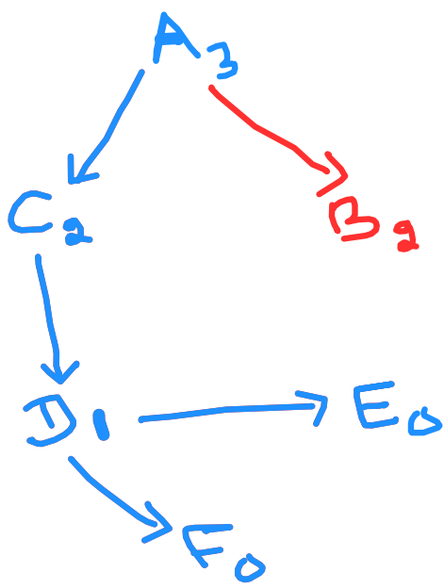
E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$
CD_1	$AB_2 \mid DE_0$, DF_0
DF_0	$AB_2 \mid DE_0$



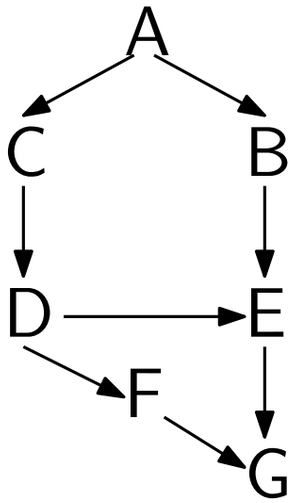
DL-DFS Tracing

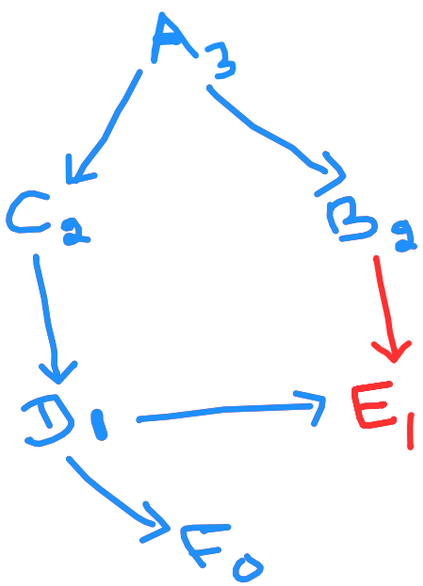


E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$
CD_1	$AB_2 \mid DE_0$, DF_0
DF_0	$AB_2 \mid DE_0$
DE_0	AB_2



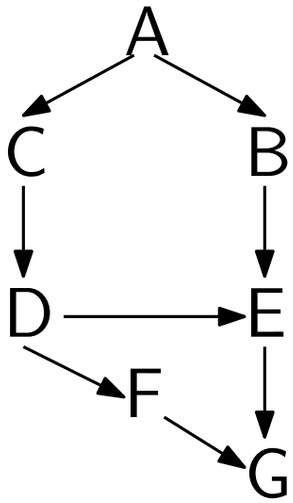
DL-DFS Tracing

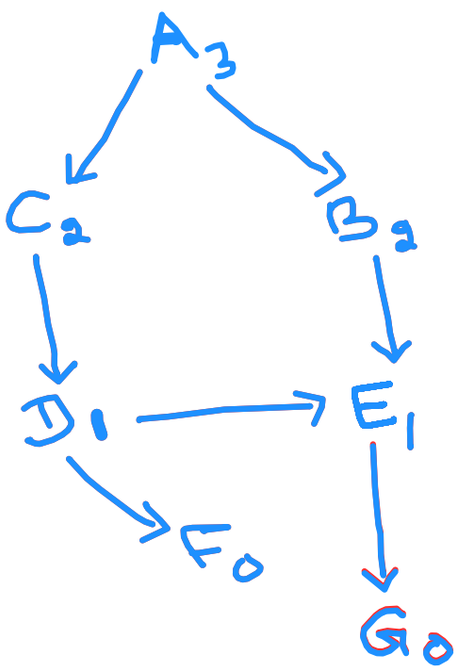




E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$
CD_1	$AB_2 \mid DE_0$, DF_0
DF_0	$AB_2 \mid DE_0$
DE_0	AB_2
AB_2	BE_1

DL-DFS Tracing





E	F
	$-A$
$-A_3$	AB_2, AC_2
AC_2	$AB_2 \mid CD_1$
CD_1	$AB_2 \mid DE_0$, DF_0
DF_0	$AB_2 \mid DE_0$
DE_0	AB_2
AB_2	BE_1
BE_1	EG_0
EG_0	

Iteratively Deepening DFS

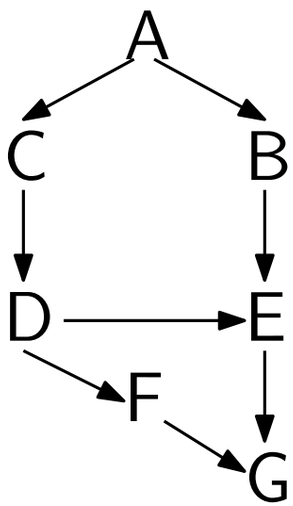
DFS with Iterative Deepening

- Space efficiency of DFS and optimality of BFS!
- Do not store the elements of BFS in frontier, but recompute!
- Recompute using DFS!
- Idea:
  - Modify DFS to stop at depth-limit.
  - Call DFS for depth-limits 1, 2, ...
    DFS to depth-limit 1: found a path?
    DFS to depth-limit 2: found a path?
    ... until a path is found.

DFS with Iterative Deepening

- Iterative-deepening search:
  - Start with a depth-limit `limit = 0`.
  - Do a depth-limited DFS with depth-limit `limit`
  - If a solution is found return that solution
  - Otherwise increment `limit` and repeat.
- This will find the same first solution as which other method?
- How much space is used?
- What happens if there is no path to a goal?
- Surely recomputing paths is wasteful!!

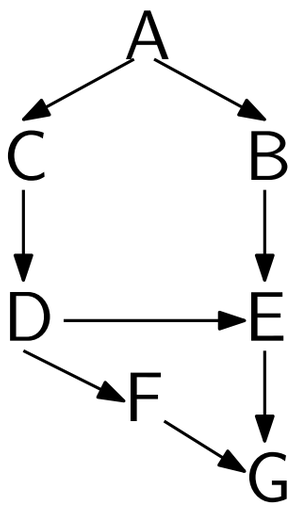
- Failure because
  - depth-limit was reached – retry
  - search space was exhausted – no more successors – fail

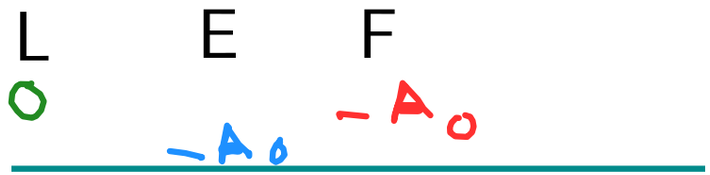


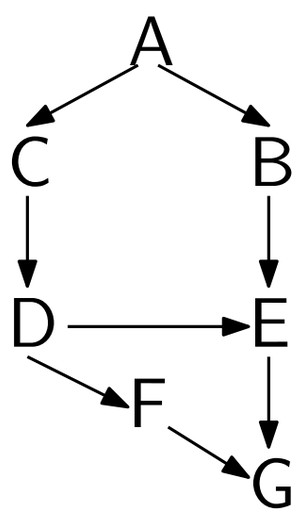
L E F

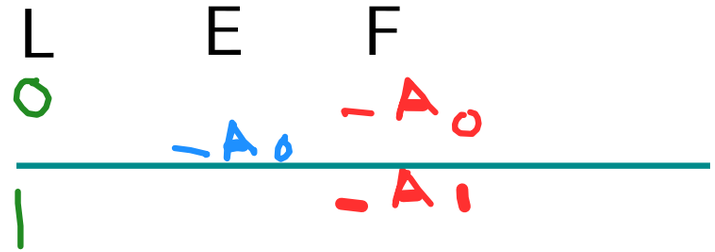
0

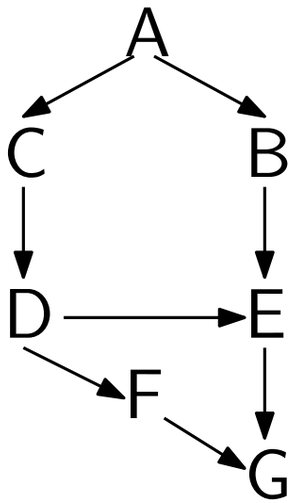
- A 0


A
C
B
D
E
F
G

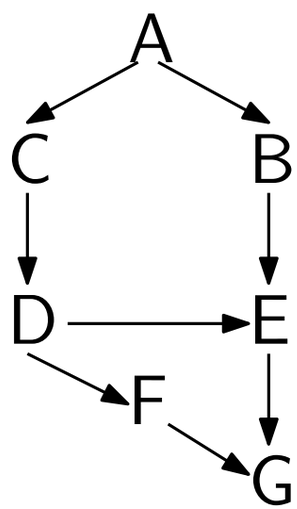

L
E
F
0
-A0
-A0


A
C
B
D
E
F
G

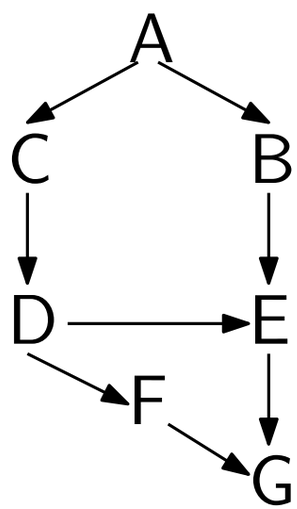

L
E
F
0
1
$-A_0$
$-A_0$
$-A_1$



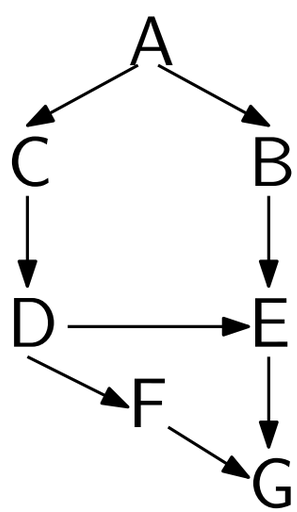
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$	$-A_1$ AB_0, AC_0



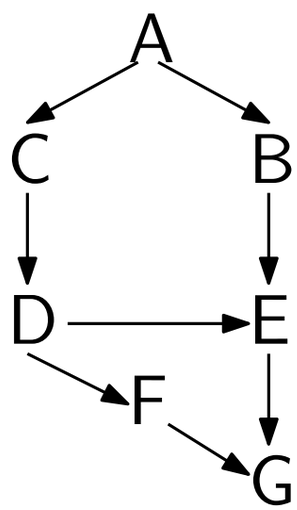
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0	$-A_1$ AB_0, AC_0 AB_0



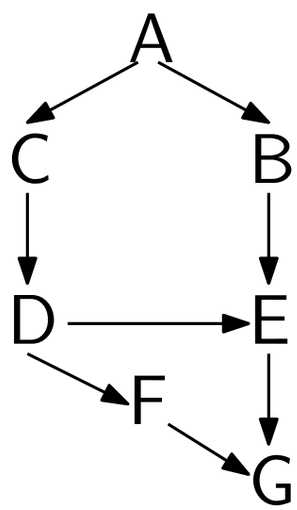
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0



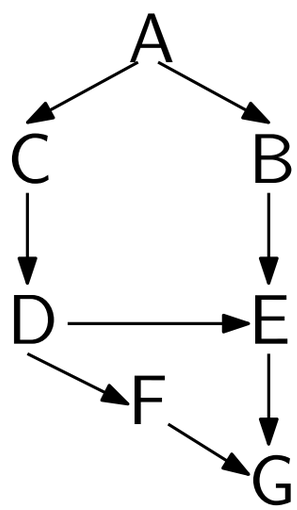
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2		$-A_2$



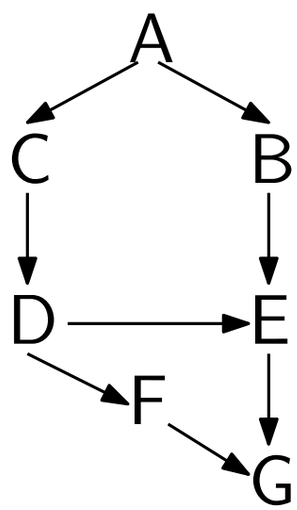
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$	$-A_2$ AB_1, AC_1



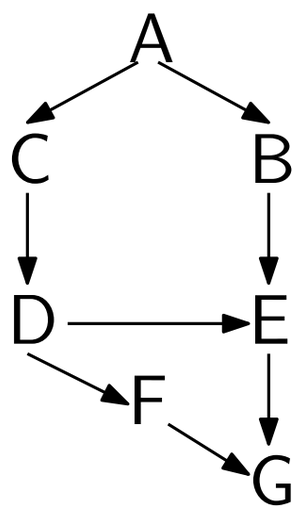
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$, AC_0, AB_0	$-A_1$, AB_0, AC_0, AB_0
2	$-A_2$, AC_1	$-A_2$, AB_1, AC_1, $AB_1 \mid CD_0$



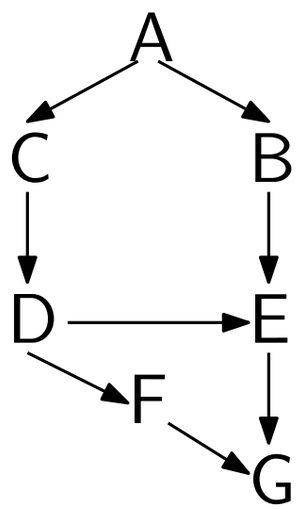
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1



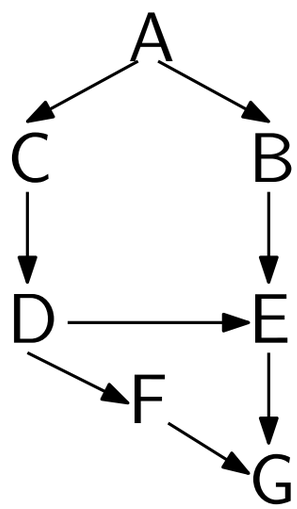
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0



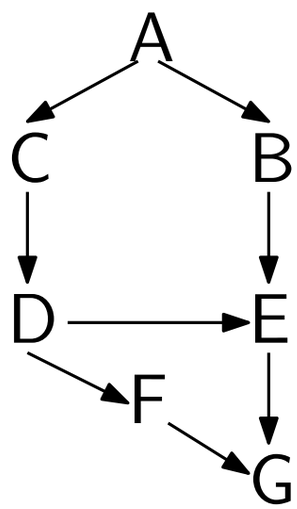
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0



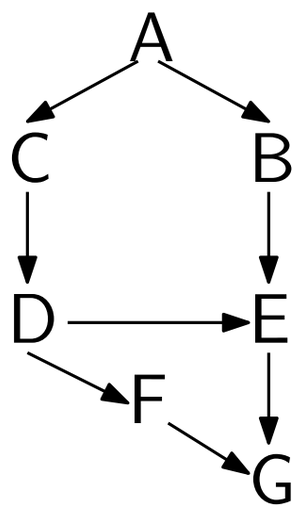
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3		$-A_3$



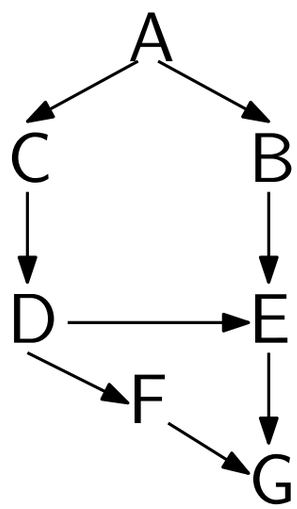
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$	$-A_3$ AB_2, AC_2



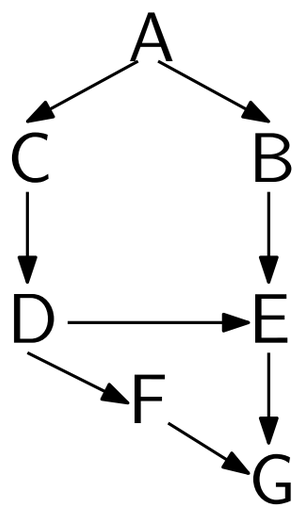
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$



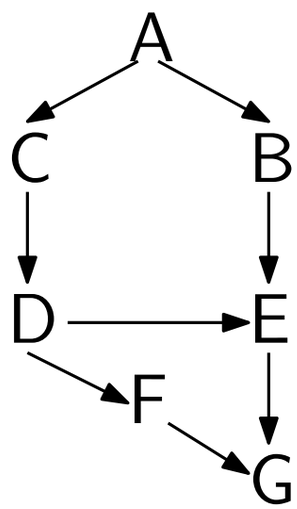
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$



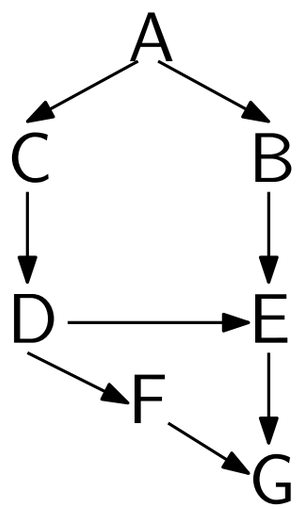
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1 DF_0	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$ $AB_2 \mid DE_0$



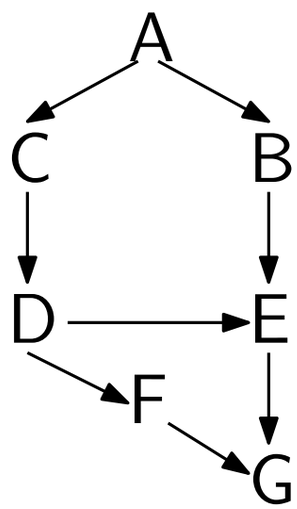
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1 DF_0 DE_0	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$ $AB_2 \mid DE_0$ AB_2



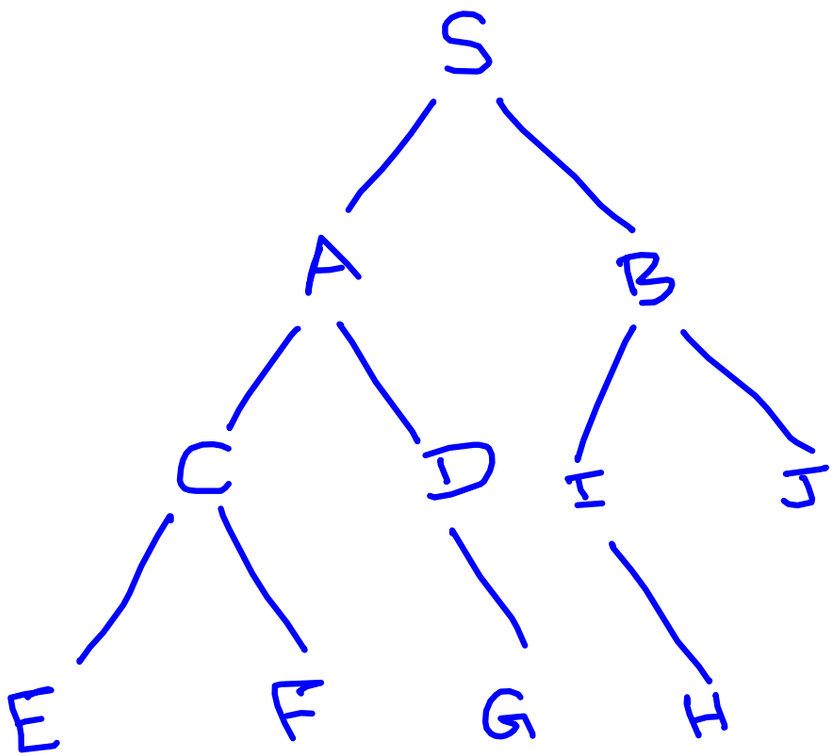
L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1 DF_0 DE_0 AB_2	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$ $AB_2 \mid DE_0$ AB_2 BE_1



L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1 DF_0 DE_0 AB_2 BE_1	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$ $AB_2 \mid DE_0$ AB_2 BE_1 EG_0



L	E	F
0	$-A_0$	$-A_0$
1	$-A_1$ AC_0 AB_0	$-A_1$ AB_0, AC_0 AB_0
2	$-A_2$ AC_1 CD_0 AB_1 BE_0	$-A_2$ AB_1, AC_1 $AB_1 \mid CD_0$ AB_1 BE_0
3	$-A_3$ AC_2 CD_1 DF_0 DE_0 AB_2 BE_1 EG_0	$-A_3$ AB_2, AC_2 $AB_2 \mid CD_1$ $AB_2 \mid DE_0, DF_0$ $AB_2 \mid DE_0$ AB_2 BE_1 EG_0



Depth	Nodes visited
0	S
1	S A B
2	S A C D B I J
3	S A C E F D G

DFS with Iterative Deepening

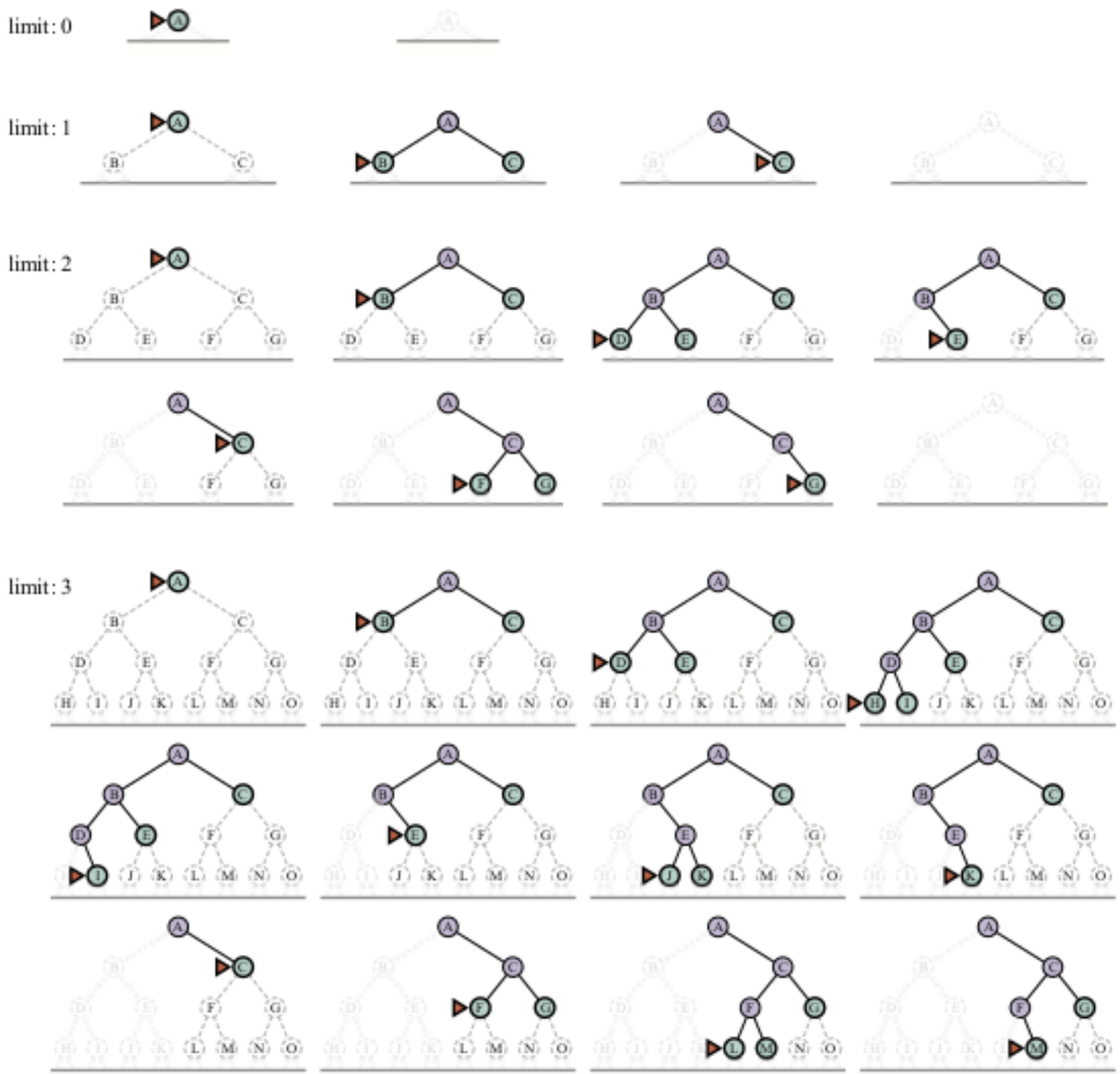
Algorithm: Iterative_Deepening_DFS(G, s, goal)

Input: G: graph
s: start node
goal: Boolean function on nodes

Output: path from s to a node for which goal is true or failure if there is no such path

```
limit ← 0
repeat
    depth_limit_hit? ← false
    result ← Depth_Limited_DFS([s], limit)
    if result ≠ failure then
        return result
    limit ← limit + 1
until not depth_limit_hit?
return failure
```

DFS with ID Trace



ID-DFS Time Complexity

Consider the search with depth-limit k & branching factor b. How many times nodes at depth k are generated?

depth	#	BFS	ID-DFS
1	b	1	k
2	b^2	1	$k-1$
...	...		
k-2	b^3	1	
k-1	b^{k-1}	1	2
k	b^k	1	1

BFS: $\dfrac{b^{k+1}-1}{b-1} = b^k(\frac{b}{b-1}) - \frac{1}{b-1} = O(b^k)$

$$b^k + 2b^{k-1} + 3b^{k-2} + \ldots + kb = b^k(1 + 2b^{-1} + 3b^{-2} + \ldots + kb^{-(k-1)}$$

$$< b^k \left(\sum_{i=1}^{\infty} \frac{i}{b^{i-1}} \right) = b^k \left(\frac{b}{b-1} \right)^2 = O(b^k)$$

Depth-First Branch-and-Bound

Depth-First Branch-and-Bound

- Combines depth-first search with heuristic information.
- Finds optimal solution.
- Most useful when there are multiple solutions, and we want an optimal one.
- Uses the space of depth-first search.

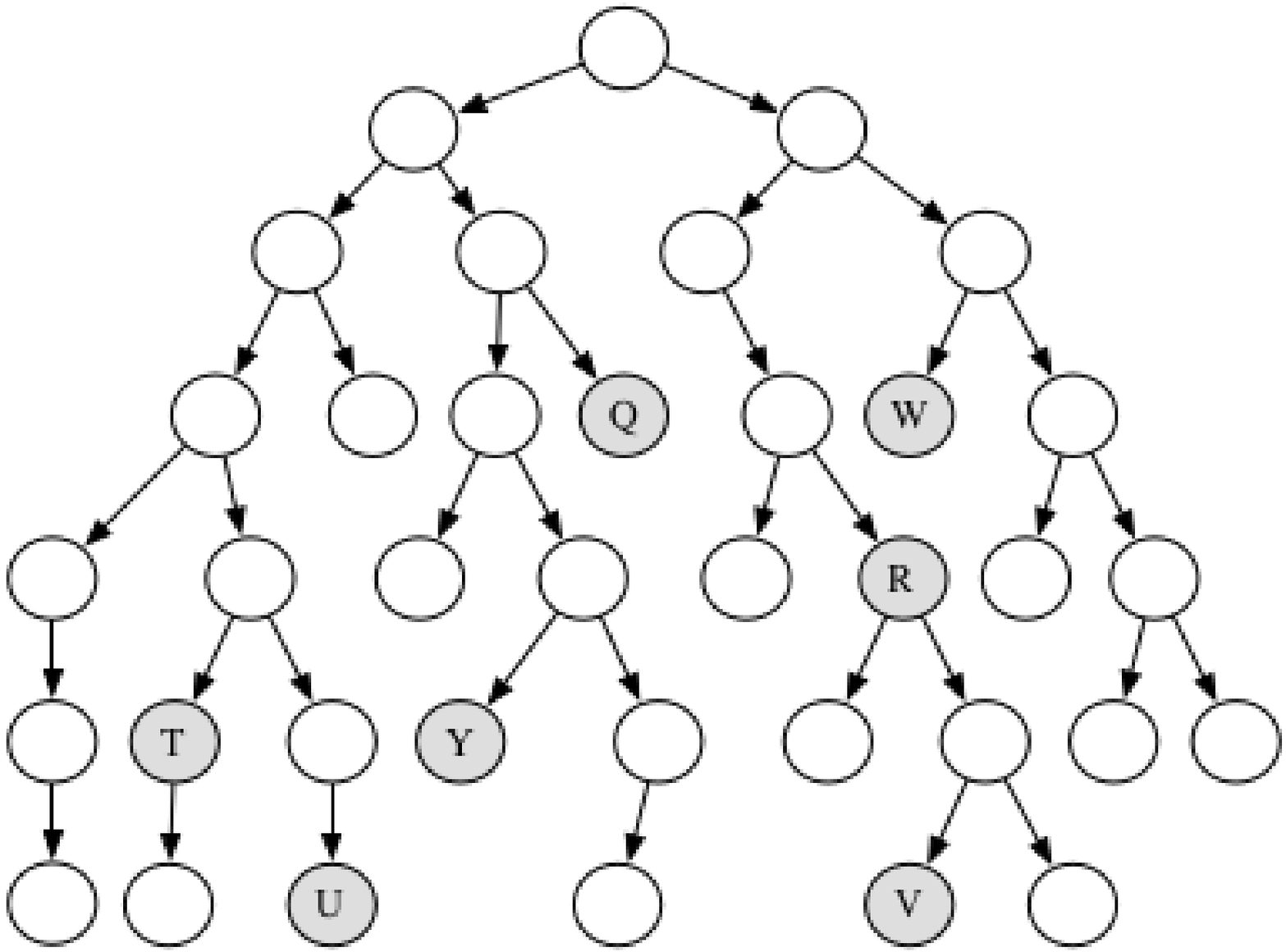
Depth-First Branch-and-Bound

- Suppose we want to find a single optimal solution.
- Suppose bound is the cost of the lowest-cost path found to a goal so far.
- What if the search encounters a path p such that $cost(p) + h(p) \geq bound$?
  p can be pruned.
- What can we do if a non-pruned path to a goal is found? bound can be set to the cost of p, and p can be remembered as the best solution so far.
- Use a depth-first search (implement similar to depth-bounded DFS)
  Uses linear space.
- What can be guaranteed when the search completes?
  It has found an optimal solution.
- How should the bound be initialized?

Depth-First Branch-and-Bound

- The bound can be initialized to ∞.
- The bound can be set to an estimate of the optimal path cost. After depth-first search terminates either:
  - A solution was found.
  - No solution was found, and no path was pruned
  - No solution was found, and a path was pruned.

Which shaded goals will be best solutions so far?



Cost-Bounded DFS

Algorithm: Cost_Bounded_DFS($[s, \ldots, v]$)

if cost($([s, \ldots, v]) + h(v) < \text{bound}$ **then**
  if goal(v) **then**
    best_path $\leftarrow [s, \ldots, v]$
    bound $\leftarrow$ cost($[s, \ldots, v]$)
  else
    foreach vw **do**
      Cost_Bounded_DFS($[s, \ldots, v, w]$)

Algorithm: Depth_Bounded_DFS([s...v], b)

Input: [s..v]: path
b: integer, b $\geq$ 0
Output: path to goal of length k + b if one exists (k = length([s...v]))

```
if b > 0 then
    foreach vw do
        res ← Depth_Bounded_DFS([s...,v,w], b - 1)
        if res ≠ failure then
            return res
else if goal(v) then // why after b > 0 test
    return [s...v]
else if v has any successors then
    hit_depth_bound ← true
return failure
```

- Use non-local variables `best_path` and `bound` to to return information to the caller.

Depth-First Branch-and-Bound

Algorithm: DF_Branch_and_Bound(G, s, goal, h, bound_0)

Input: G: graph with nodes N and arcs A
s: start node
goal: Boolean function on nodes
h: heuristic function on nodes
bound0 : initial depth bound (can be ∞ if not specified)

Output: a lowest-cost path from s to a goal node if there is a solution with cost less than bound_0 or failure if there is no solution with cost less than bound_0

best_path: path or failure
bound: non-negative real

best_path $\leftarrow$ failure
bound $\leftarrow$ bound_0
Cost_Bounded_DFS(s)
return best_path

End